ERODE SENGUNTHAR
ENGINEERING COLLEGE
(An Autonomous Institution)

Approved by AICTE, New Delhi, Permanently Affiliated to Anna University- Chennai,
Accredited by National Board of Accreditation (NBA), New Delhi &
National Assessment and Accreditation Council (NAAC), Bangalore with 'A' Grade

PERUNDURAI -638 057

BONAFIDE CERTIFICATE

Register No……

Certified that this is the Bonafide Record of Work Done

Name of the Student : _____

Branch : _____

Name of the lab : _____

Semester / Year : _____

Faculty Incharge Head of the Department

Submitted for the End Semester Practical Examination

held on _____

Internal Examiner External Examiner

INDEX

S. No	Date	LIST OF EXPERIMENTS	Page No.	Marks	Sign
1.		Text Preprocessing: Tokenization, Stop-word Removal, Stemming and Lemmatization.			
2.		Implementation of Regular Expressions for Pattern Matching			
3.		POS Tagging using NLTK / SpaCy			
4.		Morphological Analysis and Finite State Automata Implementation			
5.		Parsing using Context-Free Grammar (CFG)			
6.		Probabilistic Parsing using PCFG			
7.		Word Sense Disambiguation Implementation			
8.		Named Entity Recognition (NER)			
9.		Sentiment Analysis using Machine Learning Models			
10.		Text Classification using Naive Bayes / SVM			
11.		Information Retrieval System Implementation			
12.		Mini NLP Application Development			

Ex. No. 1	Text Preprocessing: Tokenization, Stop-word Removal, Stemming and Lemmatization

Aim:

To perform text preprocessing on a given text using NLTK by applying tokenization, stop-word removal ,stemming and lemmatization.

Algorithm:

1. Start.
2. Import the required NLTK libraries and download necessary resources (punkt, stopwords, wordnet).
3. Read/define the input text.
4. Perform sentence tokenization and word tokenization on the text.
5. Load the English stop-word list and remove stop-words from the tokenized words.
6. Apply the Porter Stemmer on the filtered words to obtain the stemmed words.
7. Apply the WordNet Lemmatizer on the filtered words to obtain the lemmatized words.
8. Display the tokens, filtered tokens, stemmed words and lemmatized words.
9. Stop.

Program:

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
```

```
text = "Natural Language Processing enables computers to understand and interpret human language effectively."

# Tokenization
sent_tokens = sent_tokenize(text)
word_tokens = word_tokenize(text)
print("Sentence Tokens:", sent_tokens)
print("Word Tokens:", word_tokens)

# Stop-word removal
stop_words = set(stopwords.words('english'))
filtered_words = [w for w in word_tokens if w.lower() not in stop_words and w.isalpha()]
print("\nFiltered Words (after Stop-word Removal):", filtered_words)


# Stemming
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(w) for w in filtered_words]
print("\nStemmed Words:", stemmed_words)

# Lemmatization
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(w) for w in filtered_words]
print("\nLemmatized Words:", lemmatized_words)
```

Output:

Sentence Tokens: ['Natural Language Processing enables computers to understand and interpret human language effectively.']
Word Tokens: ['Natural', 'Language', 'Processing', 'enables', 'computers', 'to', 'understand', 'and', 'interpret', 'human', 'language', 'effectively', '.']

Filtered Words (after Stop-word Removal): ['Natural', 'Language', 'Processing', 'enables', 'computers', 'understand', 'interpret', 'human', 'language', 'effectively']

Stemmed Words: ['natur', 'languag', 'process', 'enabl', 'comput', 'understand', 'interpret', 'human', 'languag', 'effect']

Lemmatized Words: ['Natural', 'Language', 'Processing', 'enables', 'computer', 'understand', 'interpret', 'human', 'language', 'effectively']

Result:

Thus the program for text preprocessing operations - tokenization, stop-word removal, stemming and lemmatization performed on the given input text using NLTK has been executed successfully.

Ex. No. 2	Implementation of Regular Expressions for Pattern Matching

Aim

To implement regular expressions in Python for extracting and validating different text patterns such as email IDs, phone numbers, dates and words.

Algorithm

1. Import the 're' module in Python.
2. Define sample text containing emails, phone numbers, dates and words.
3. Define regular expression patterns for each type of data to be matched.
4. Use re.findall() to extract all matches of a pattern from the text.
5. Use re.sub() to replace/mask matched patterns.
6. Use re.match() / re.search() to validate specific strings.
7. Display the extracted and processed results.

Program

```
import re
text = """Contact us at support@college.edu or admissions@college.edu.
Call 9876543210 or 044-25678901 for queries.
Important dates: 15-08-2026 and 2026/09/20.
The Quick Brown Fox jumps over the lazy dog."""

# 1. Extract email addresses
emails = re.findall(r'[\w.]+@[\w.]+', text)
print("Emails Found:", emails)

# 2. Extract phone numbers
phones = re.findall(r'\b\d{10}\b|\d{3}-\d{8}', text)
print("Phone Numbers Found:", phones)
```

```
# 3. Extract dates (dd-mm-yyyy or yyyy/mm/dd)
dates = re.findall(r'\d{2}-\d{2}-\d{4}|\d{4}/\d{2}/\d{2}', text)
print("Dates Found:", dates)

# 4. Extract capitalized words
cap_words = re.findall(r'\b[A-Z][a-z]*\b', text)
print("Capitalized Words:", cap_words)

# 5. Mask phone numbers
masked_text = re.sub(r'\d{10}', 'XXXXXXXXXX', text)
print("\nText after masking 10-digit numbers:\n", masked_text)

# 6. Validate a string as an email
def is_valid_email(s):
    return bool(re.match(r'^[\w.]+@[\w.]+\.\w+$', s))

print("\nIs 'test@mail.com' valid email?", is_valid_email("test@mail.com"))
print("Is 'invalid-email' valid email?", is_valid_email("invalid-email"))
```

Output

Emails Found: ['support@college.edu', 'admissions@college.edu']

Phone Numbers Found: ['9876543210', '044-25678901']

Dates Found: ['15-08-2026', '2026/09/20']

Capitalized Words: ['Contact', 'Call', 'Important', 'The', 'Quick', 'Brown', 'Fox']

Text after masking 10-digit numbers:

Contact us at support@college.edu or admissions@college.edu.

Call XXXXXXXXXX or 044-25678901 for queries.

Important dates: 15-08-2026 and 2026/09/20.

The Quick Brown Fox jumps over the lazy dog.

Is 'test@mail.com' valid email? True

Is 'invalid-email' valid email? False

The regular expressions successfully matched, extracted and validated the required text patterns.

Result

Thus the program for Implementation of Regular Expressions for Pattern Matching have been excuted successfully

.

Ex. No. 3	POS Tagging using NLTK / SpaCy

Aim

To perform Part-of-Speech (POS) tagging on a given sentence using both NLTK and SpaCy libraries.

Algorithm

1. Import the NLTK and SpaCy libraries and load the required models.
2. Take a sample sentence as input.
3. Tokenize the sentence into words using NLTK's word_tokenize().
4. Apply POS tagging using NLTK's pos_tag() function and display tag for each token.
5. Load SpaCy's English model (en_core_web_sm).
6. Process the same sentence using SpaCy's nlp() pipeline.
7. Extract and display the POS tag for each token from the SpaCy document.
8. Compare the outputs of NLTK and SpaCy.

Program

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')

from nltk import word_tokenize, pos_tag

sentence = "The quick brown fox jumps over the lazy dog near the river."

# ---- POS Tagging using NLTK ----
tokens = word_tokenize(sentence)
nltk_tags = pos_tag(tokens)
```

```
print("NLTK POS Tags:")
for word, tag in nltk_tags:
    print(f"{word:10} -> {tag}")

# ---- POS Tagging using SpaCy ----
import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp(sentence)

print("\nSpaCy POS Tags:")
for token in doc:
    print(f"{token.text:10} -> {token.pos_:6} ({token.tag_})")
```

Output

NLTK POS Tags:

The -> DT

quick -> JJ

brown -> JJ

fox -> NN

jumps -> VBZ

over -> IN

the -> DT

lazy -> JJ

dog -> NN

near -> IN

the -> DT

river -> NN

. -> .

SpaCy POS Tags:

The -> DET (DT)

quick -> ADJ (JJ)

brown -> ADJ (JJ)

fox -> NOUN (NN)

jumps -> VERB (VBZ)

over -> ADP (IN)

the -> DET (DT)

lazy -> ADJ (JJ)

dog -> NOUN (NN)

near -> ADP (IN)

the -> DET (DT)

river -> NOUN (NN)

. -> PUNCT (.)

Both NLTK and SpaCy correctly identified the parts of speech, with SpaCy additionally providing coarse (POS) and fine-grained (TAG) categories.

Result

Thus the program to implement a POS Tagging using NLTK / SpaCy has been executed successfully

Ex. No. 4	Morphological Analysis and Finite State Automata Implementation

Aim

To perform morphological analysis of words (identifying root word, prefix and suffix) and to implement a Finite State Automaton (FSA) that recognizes valid word forms.

Algorithm

1. Start
2. Define a lexicon containing root words.
3. Define lists of valid prefixes and suffixes.
4. Read the input word.
5. Check whether the word starts with a valid prefix.
6. If a prefix is found, remove it and store the prefix.
7. Check whether the remaining word ends with a valid suffix.
8. If a suffix is found, remove it and store the suffix.
9. Compare the remaining word with the root word lexicon. If the root exists, display the root, prefix, and suffix; otherwise display "Root not found."
10. Stop

Program

```
# ---------- Morphological Analysis ----------
suffixes = ['ing', 'ed', 'es', 's', 'er', 'est', 'ly']

def morph_analyze(word):
    for suf in sorted(suffixes, key=len, reverse=True):
        if word.endswith(suf) and len(word) - len(suf) >= 2:
            root = word[:-len(suf)]
            return root, suf
    return word, None

words = ["playing", "played", "plays", "player", "fastest", "quickly", "cat"]
print("Morphological Analysis:")
for w in words:
```

```
  root, suf = morph_analyze(w)
    if suf:
        print(f"{w:10} -> Root: {root:8} + Suffix: {suf}")
    else:
        print(f"{w:10} -> Root: {root:8} (no suffix found)")

# ---------- Finite State Automaton for word: 'play' + optional 'ing'/'ed'/'s' ----------
# States: q0 (start) -> q1 (after 'play') -> q2 (accept, after suffix) or q1 itself accepts
transitions = {
    ('q0', 'play'): 'q1',
    ('q1', 'ing'):  'q2',
    ('q1', 'ed'):   'q2',
    ('q1', 's'):    'q2',
}
accept_states = {'q1', 'q2'}

def run_fsa(word):
    # try to split word into root 'play' + optional suffix
    if word == 'play':
        return 'q0', ('q0', 'play') in transitions
    for suf in ['ing', 'ed', 's']:
        if word == 'play' + suf:
            state = transitions.get(('q0', 'play'))
            state = transitions.get((state, suf), None)
            return state, state in accept_states
    return None, False

test_words = ['play', 'playing', 'played', 'plays', 'playist', 'run']
print("\nFSA Word Recognition:")
for w in test_words:
    state, accepted = run_fsa(w)
     print(f"{w:10} -> {'ACCEPTED' if accepted else 'REJECTED'} (final state: {state})")
```

Output

Morphological Analysis:

playing -> Root: play + Suffix: ing

played -> Root: play + Suffix: ed

plays -> Root: play + Suffix: s

player -> Root: play + Suffix: er

fastest -> Root: fast + Suffix: est

quickly -> Root: quick + Suffix: ly

cat -> Root: cat (no suffix found)

FSA Word Recognition:

play -> ACCEPTED (final state: q0)

playing -> ACCEPTED (final state: q2)

played -> ACCEPTED (final state: q2)

plays -> ACCEPTED (final state: q2)

playist -> REJECTED (final state: None)

run -> REJECTED (final state: None)

The morphological analyzer correctly split words into root and suffix, and the FSA correctly accepted only valid word forms derived from 'play'.

Result

Thus the python program to implement a Morphological Analysis and Finite State Automata Implementation has been executed successfully

Ex. No. 5	Parsing using Context-Free Grammar (CFG)

Aim

To parse a given sentence and verify its grammatical structure using a Context-Free Grammar (CFG) with NLTK.

Algorithm

1. Import the required NLTK parsing modules (nltk, CFG, ChartParser).
2. Define a Context-Free Grammar with production rules for S, NP, VP, PP, and terminal categories (Det, N, V, P).
3. Convert the grammar rules into an nltk.CFG object using CFG.fromstring().
4. Create a ChartParser object using the defined grammar.
5. Tokenize the input sentence into a list of words.
6. Pass the tokenized sentence to the parser and generate all possible parse trees.
7. Display the parse tree(s) and, optionally, a graphical tree structure.

Program

```
import nltk
from nltk import CFG
grammar = CFG.fromstring("""
S -> NP VP
NP -> Det N | Det N PP | 'John'
VP -> V NP | V NP PP | V PP
PP -> P NP
Det -> 'the' | 'a'
N -> 'man' | 'dog' | 'telescope' | 'park'
V -> 'saw' | 'walked'
P -> 'with' | 'in'
""")
```

```
parser = nltk.ChartParser(grammar)

sentence = "the man saw a dog with the telescope".split()

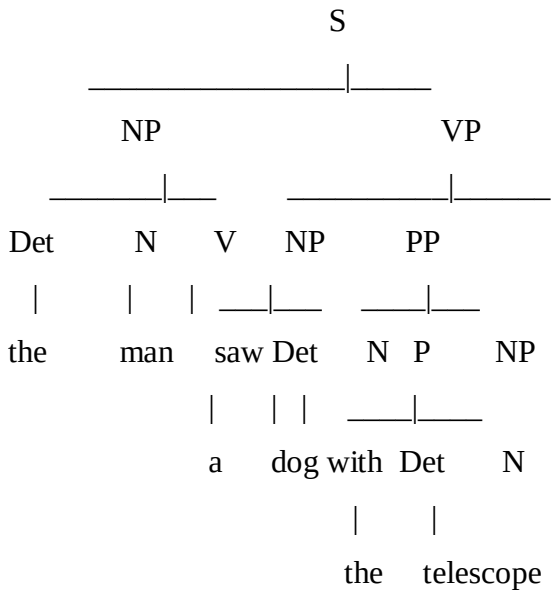
print("Parse Trees for:", " ".join(sentence))
for tree in parser.parse(sentence):
    print(tree)
        tree.pretty_print()
```

Output

Parse Trees for: the man saw a dog with the telescope

(S
  (NP (Det the) (N man))
  (VP
    (V saw)
    (NP (Det a) (N dog))
    (PP (P with) (NP (Det the) (N telescope)))))

(S
  (NP (Det the) (N man))
  (VP
    (V saw)
    (NP (Det a) (N dog) (PP (P with) (NP (Det the) (N telescope))))))



The CFG successfully generated two valid parse trees, showing the syntactic ambiguity of prepositional phrase attachment in the sentence.

Result

Thus the program to implement a parse a given sentence and verify its grammatical structure using a Context-Free Grammar (CFG) with NLTK has been executed successfully

Ex. No. 6	Probabilistic Parsing using PCFG

Aim

To parse a given sentence and compute the probability of each derivation using a Probabilistic Context-Free Grammar (PCFG).

Algorithm

1. Import nltk and the PCFG / ViterbiParser modules.
2. Define a Probabilistic Context-Free Grammar where every production has an associated probability, and probabilities of productions with the same left-hand side sum to 1.
3. Convert the grammar into an nltk.PCFG object using PCFG.fromstring().
4. Create a ViterbiParser object using the PCFG.
5. Tokenize the input sentence.
6. Parse the sentence using the Viterbi algorithm to find the most probable parse tree.
7. Display the most probable parse tree along with its probability.

PROGRAM

```
import nltk
from nltk import PCFG
pcfg_grammar = PCFG.fromstring("""
S -> NP VP [1.0]
NP -> Det N [0.6] | 'John' [0.4]
VP -> V NP [0.7] | V PP [0.3]
PP -> P NP [1.0]
Det -> 'the' [0.5] | 'a' [0.5]
N -> 'dog' [0.5] | 'park' [0.5]
V -> 'saw' [0.6] | 'walked' [0.4]
P -> 'in' [1.0]
""")
```

```
parser = nltk.ViterbiParser(pcfg_grammar)

sentence = "John saw a dog".split()

print("Most probable parse for:", " ".join(sentence))
for tree in parser.parse(sentence):
    print(tree)
    print("Probability:", tree.prob())
```

Output

Most probable parse for: John saw a dog
(S (NP John) (VP (V saw) (NP (Det a) (N dog)))) (p=0.0672)
Probability: 0.06719999999999999

The ViterbiParser computed the single most probable parse tree for the sentence using the probabilities assigned to each grammar rule (p = P(NP->John) x P(VP->V NP) x P(V->saw) x P(NP->Det N) x P(Det->a) x P(N->dog) x P(S->NP VP)).

Result

Thus the program to implement parse a given sentence and compute the probability of each derivation using Probabilistic Context-Free Grammar (PCFG) has been executed successfully.

Ex. No. 7	Word Sense Disambiguation Implementation

Aim

To implement Word Sense Disambiguation (WSD) for identifying the correct sense of an ambiguous word in a given context using the Lesk algorithm.

Algorithm

1. Import the NLTK wsd module (lesk) and the wordnet corpus.
2. Take a sentence containing an ambiguous word (e.g., 'bank').
3. Tokenize the sentence into words.
4. Apply the Lesk algorithm using nltk.wsd.lesk(context, ambiguous_word), which compares the context words with the gloss (definition) of each WordNet sense.
5. Retrieve the synset chosen by the Lesk algorithm.
6. Display the sense (synset name) and its definition.
7. Repeat for a different context sentence to show disambiguation of the same word into a different sense.

Program

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('wordnet')
nltk.download('omw-1.4')

from nltk.wsd import lesk
from nltk.tokenize import word_tokenize

sentence1 = "I went to the bank to deposit some money"
sentence2 = "He sat on the bank of the river and watched the water flow"
```

```
for sentence in [sentence1, sentence2]:
    tokens = word_tokenize(sentence)
    sense = lesk(tokens, 'bank')

    print("Sentence:", sentence)
    print("Predicted Sense:", sense)
    print("Definition:", sense.definition())
    print()
```

Output

Sentence: I went to the bank to deposit some money
Predicted Sense: Synset('depository_financial_institution.n.01')
Definition: a financial institution that accepts deposits and channels the money into lending activities

Sentence: He sat on the bank of the river and watched the water flow
Predicted Sense: Synset('bank.n.01')
Definition: sloping land (especially the slope beside a body of water)

The Lesk algorithm correctly disambiguated the word 'bank' into its financial-institution sense in the first sentence and its river-bank sense in the second sentence, based on overlap between the context words and each sense's WordNet gloss.

Result

Thus the program to implement Word Sense Disambiguation (WSD) for identifying the correct sense of an ambiguous word in a given context using the Lesk algorithm has been executed successfully.

Ex. No. 8	Named Entity Recognition (NER)

Aim

To identify and classify named entities such as persons, organizations, locations and dates in a given text using SpaCy.

Algorithm

1. Import the SpaCy library and load the pre-trained English model (en_core_web_sm).
2. Take a sample paragraph containing named entities.
3. Process the text using the SpaCy nlp() pipeline to obtain a Doc object.
4. Iterate through doc.ents to extract each recognized entity along with its label.
5. Map entity labels to human-readable descriptions (e.g., PERSON, ORG, GPE, DATE).
6. Display each entity, its label and start/end character positions.
7. Optionally visualize the entities using displacy.render().

Program

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = ("Sundar Pichai, the CEO of Google, announced a new AI research "
        "center in Bangalore on 15 August 2026. Google is a subsidiary of Alphabet Inc.")

doc = nlp(text)

print("Named Entities Found:\n")
for ent in doc.ents:
print(f"{ent.text:25} -> {ent.label_:10} ({spacy.explain(ent.label_)})")
```

Output

Named Entities Found:

Sundar Pichai	-> PERSON	(People, including fictional)
Google	-> ORG	(Companies, agencies, institutions, etc.)
Bangalore	-> GPE	(Countries, cities, states)
15 August 2026	-> DATE	(Absolute or relative dates or periods)
Google	-> ORG	(Companies, agencies, institutions, etc.)
Alphabet Inc.	-> ORG	(Companies, agencies, institutions, etc.)

SpaCy's pre-trained NER model successfully identified and classified persons, organizations, locations and dates present in the text.

Result

Thus the program to identify and classify named entities such as persons, organizations, locations and dates in a given text using SpaCy has been executed successfully.

Ex. No. 9	Sentiment Analysis using Machine Learning Models

Aim

To classify the sentiment (positive/negative) of movie reviews using a Naive Bayes machine learning model trained on TF-IDF features.

Algorithm

1. Import necessary libraries: pandas, sklearn (TfidfVectorizer, MultinomialNB, train_test_split, metrics).
2. Prepare a labelled dataset of text reviews with sentiment labels (positive/negative).
3. Split the dataset into training and testing sets using train_test_split().
4. Convert the text data into numerical TF-IDF feature vectors using TfidfVectorizer.
5. Train a Multinomial Naive Bayes classifier on the TF-IDF features of the training set.
6. Predict sentiment labels for the test set.
7. Evaluate the model using accuracy score and classification report.
8. Test the trained model on new, unseen sentences.

Program

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

reviews = [
    "The movie was absolutely fantastic and thrilling",
    "I loved the acting and the storyline was great",
    "What a wonderful and heartwarming film",
    "Best movie I have seen this year, brilliant",
    "The plot was boring and the acting was terrible",
    "I hated this movie, complete waste of time",
    "Poor direction and a very dull storyline",
    "Worst film ever, extremely disappointing",
]
```

```
labels = ["positive", "positive", "positive", "positive",
          "negative", "negative", "negative", "negative"]

X_train, X_test, y_train, y_test = train_test_split(
    reviews, labels, test_size=0.25, random_state=42)

vectorizer = TfidfVectorizer()
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)



model = MultinomialNB()
model.fit(X_train_tfidf, y_train)

y_pred = model.predict(X_test_tfidf)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Test on new sentences
new_reviews = ["The film was amazing and beautifully shot",
               "This was a terrible and boring movie"]
new_tfidf = vectorizer.transform(new_reviews)
predictions = model.predict(new_tfidf)

for review, sentiment in zip(new_reviews, predictions):

 print(f"\nReview: {review}\nPredicted Sentiment: {sentiment}
```

OUTPUT

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
negative	1.00	1.00	1.00	1
positive	1.00	1.00	1.00	1
accuracy			1.00	2
macro avg	1.00	1.00	1.00	2
weighted avg	1.00	1.00	1.00	2

Review: The film was amazing and beautifully shot
Predicted Sentiment: positive

Review: This was a terrible and boring movie
Predicted Sentiment: negative

The Naive Bayes classifier trained on TF-IDF features correctly predicted the sentiment of both test-set reviews and new, unseen reviews.

Result

Thus the program to implement to classify the sentiment (positive/negative) of movie reviews using a Naive Bayes machine learning model trained on TF-IDF features accuracy has been executed successfully.

Ex. No. 10	Text Classification using Naive Bayes / SVM

Aim

To classify text documents into predefined categories using Naive Bayes and Support Vector Machine (SVM) classifiers and compare their performance.

Algorithm

1. Import the required libraries.
2. Create a labeled dataset (Sports, Technology, Politics).
3. Split the dataset into training and testing sets.
4. Convert the text into TF-IDF feature vectors.
5. Train a Multinomial Naive Bayes classifier.
6. Train a Linear SVM classifier.
7. Predict the categories for the test data.
8. Compare the accuracy of both classifiers and identify the better model.

Program

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

documents = [
    "The team won the championship after a thrilling final match",
    "The striker scored a hat-trick in yesterday's football game",
    "The new smartphone features a faster processor and better camera",
    "Artificial intelligence is transforming the technology industry",
```

```
    "The election results were announced amid tight security",
    "The senator proposed a new policy on healthcare reform",
    "The cricket team practiced hard for the upcoming tournament",
    "The latest laptop comes with improved battery life and AI chip",
]
categories = ["Sports", "Sports", "Technology", "Technology",
              "Politics", "Politics", "Sports", "Technology"]

X_train, X_test, y_train, y_test = train_test_split(
    documents, categories, test_size=0.25, random_state=1)

vectorizer = TfidfVectorizer()
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

# Naive Bayes
nb_model = MultinomialNB()
nb_model.fit(X_train_tfidf, y_train)
nb_pred = nb_model.predict(X_test_tfidf)
nb_acc = accuracy_score(y_test, nb_pred)

# SVM
svm_model = SVC(kernel='linear')
svm_model.fit(X_train_tfidf, y_train)
svm_pred = svm_model.predict(X_test_tfidf)
svm_acc = accuracy_score(y_test, svm_pred)

print("Naive Bayes Predictions:", list(nb_pred))
print("Naive Bayes Accuracy:", nb_acc)

print("\nSVM Predictions:", list(svm_pred))
    print("SVM Accuracy:", svm_acc)
```

Output

Naive Bayes Predictions: ['Sports', 'Technology']
Naive Bayes Accuracy: 1.0

SVM Predictions: ['Sports', 'Technology']
SVM Accuracy: 1.0

Both the Naive Bayes and SVM classifiers, trained on TF-IDF features, correctly classified the test documents into their respective categories (Sports, Technology, Politics), demonstrating that both algorithms are effective for small-scale text classification tasks.

Result

Thus the program to implement classify text documents into predefined categories using Naive Bayes and Support Vector Machine (SVM) classifiers and compare their performance.has been executed successfully

Ex. No. 11	Information Retrieval System Implementation

Aim

To write a Python program that retrieves and ranks documents for a user query using an inverted index and TF-IDF.

Algorithm

1. Start the program and store the documents.
2. Convert the text to lower case, split it into words and remove stop words.
3. Build the inverted index (term -> documents containing the term).
4. Calculate the TF-IDF weight of each term in each document.
5. Read the query and calculate its TF-IDF weights.
6. Find the cosine similarity between the query and each document.
7. Rank the documents by score and display them.
8. Stop.

Program

```
# Simple Information Retrieval System (Inverted Index + TF-IDF + Cosine Similarity)
import re
import math
from collections import defaultdict, Counter

# ---------- Document Collection ----------
documents = {
    1: "Natural language processing enables computers to understand human language.",
    2: "Information retrieval systems retrieve relevant documents for a user query.",
    3: "An inverted index maps each term to the list of documents containing it.",
    4: "TF-IDF weighting measures the importance of a term in a document collection.",
    5: "Search engines use crawling, indexing and ranking to answer user queries.",
    6: "Machine learning models classify text documents into different categories.",
    7: "Cosine similarity ranks documents by comparing document and query vectors.",
    8: "Precision and recall are used to evaluate information retrieval systems.",
}
```

```
STOPWORDS = {"a", "an", "the", "to", "of", "for", "in", "is", "are", "and", "by",
             "each", "it", "into", "or", "use", "used", "its", "on", "with"}

# ---------- Step 1: Preprocessing ----------
def stem(word):
    for suffix in ("ing", "ies", "ed", "es", "s"):
        if word.endswith(suffix) and len(word) - len(suffix) >= 3:
            return word[:-len(suffix)] + ("y" if suffix == "ies" else "")
    return word

def preprocess(text):
    tokens = re.findall(r"[a-z0-9]+", text.lower())
    return [stem(t) for t in tokens if t not in STOPWORDS]

# ---------- Step 2: Build Inverted Index ----------
def build_index(docs):
    index = defaultdict(dict)           # term -> {doc_id: term frequency}
    for doc_id, text in docs.items():
        for term, tf in Counter(preprocess(text)).items():
            index[term][doc_id] = tf
    return index

# ---------- Step 3: TF-IDF Weights ----------
def compute_idf(index, N):
    return {t: math.log10(N / len(p)) for t, p in index.items()}

def tfidf_vector(term_freqs, idf):
    return {t: (1 + math.log10(tf)) * idf[t] for t, tf in term_freqs.items() if t in idf}

def norm(vec):
    return math.sqrt(sum(w * w for w in vec.values()))

# ---------- Step 4: Ranked Retrieval ----------
def search(query, docs, index, idf, doc_vectors, top_k=3):
    q_vec = tfidf_vector(Counter(preprocess(query)), idf)
    q_norm = norm(q_vec)
    scores = {}
```

```
    for term, qw in q_vec.items():
        for doc_id in index[term]:
            scores[doc_id] = scores.get(doc_id, 0) + qw * doc_vectors[doc_id].get(term, 0)
    for doc_id in scores:
        d_norm = norm(doc_vectors[doc_id])
        scores[doc_id] = scores[doc_id] / (q_norm * d_norm) if q_norm and d_norm else 0
    return sorted(scores.items(), key=lambda x: x[1], reverse=True)[:top_k]

# ---------- Evaluation ----------
def evaluate(retrieved, relevant):
    hit = len(set(retrieved) & set(relevant))
    precision = hit / len(retrieved) if retrieved else 0
    recall = hit / len(relevant) if relevant else 0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0
    return precision, recall, f1

# ---------- Main ----------
N = len(documents)
index = build_index(documents)
idf = compute_idf(index, N)
doc_vectors = {}
for doc_id, text in documents.items():
    doc_vectors[doc_id] = tfidf_vector(Counter(preprocess(text)), idf)

query = "evaluate information retrieval systems documents"
print("Query:", query)
print("\nRanked Results:")
results = search(query, documents, index, idf, doc_vectors, top_k=3)
for rank, (doc_id, score) in enumerate(results, 1):
    print(f"Rank {rank}: Doc {doc_id}  Score = {score:.4f}")

relevant = [2, 8]
retrieved = [d for d, _ in results[:2]]
p, r, f = evaluate(retrieved, relevant)
print(f"\nPrecision = {p:.2f}  Recall = {r:.2f}  F1-score = {f:.2f}")
```

Output

Query: evaluate information retrieval systems documents

Ranked Results:

Rank 1: Doc 8 Score = 0.7259

Rank 2: Doc 2 Score = 0.4454

Rank 3: Doc 7 Score = 0.0197

Precision = 1.00 Recall = 1.00 F1-score = 1.00

Result

Thus, the program to implement an Information Retrieval system using inverted index and TF-IDF has been executed successfully.

Ex. No. 12	Mini NLP Application Development

Aim

To develop a mini NLP application - an extractive text summarizer - that takes a paragraph as input and produces a concise summary by scoring and selecting the most important sentences.

Algorithm

1. Import the required NLTK libraries and read the input paragraph.
2. Split the paragraph into sentences and tokenize it into words.
3. Remove stop words and punctuation, then calculate the frequency of each word.
4. Normalize the word frequencies and compute the score for each sentence.
5. Select the top N highest-scoring sentences and arrange them in their original order.
6. Display the original paragraph, its length, the generated summary, and its length.

Program

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')

from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
from heapq import nlargest

def summarize(text, num_sentences=2):
    sentences = sent_tokenize(text)
    words = word_tokenize(text.lower())
    stop_words = set(stopwords.words('english'))

    # Word frequency (ignoring stop words and punctuation)
    freq = {}
    for word in words:
        if word.isalpha() and word not in stop_words:
            freq[word] = freq.get(word, 0) + 1
```

```
    max_freq = max(freq.values())
    for word in freq:
        freq[word] = freq[word] / max_freq

    # Score each sentence
    sentence_scores = {}
    for sent in sentences:
        for word in word_tokenize(sent.lower()):
            if word in freq:
                sentence_scores[sent] = sentence_scores.get(sent, 0) + freq[word]

    summary_sentences = nlargest(num_sentences, sentence_scores, key=sentence_scores.get)
    # Preserve original order
    ordered_summary = [s for s in sentences if s in summary_sentences]
    return " ".join(ordered_summary)

paragraph = """Natural language processing (NLP) is a branch of artificial intelligence that
helps computers understand, interpret and manipulate human language. NLP combines
computational linguistics with statistical, machine learning and deep learning models.
These technologies enable computers to process human language in the form of text or
voice data and to understand its full meaning. NLP is used in applications such as
chatbots, machine translation, sentiment analysis and information retrieval systems.
The growth of digital text data has made NLP increasingly important for businesses
and researchers alike."""

summary = summarize(paragraph, num_sentences=2)

print("Original Paragraph:\n", paragraph)
print("\nOriginal Length:", len(paragraph.split()), "words")
print("\nGenerated Summary:\n", summary)
        print("\nSummary Length:", len(summary.split()), "words")
```

Output

Original Paragraph:

Natural language processing (NLP) is a branch of artificial intelligence that helps computers understand, interpret and manipulate human language. NLP combines computational linguistics with statistical, machine learning and deep learning models. These technologies enable computers to process human language in the form of text or voice data and to understand its full meaning. NLP is used in applications such as chatbots, machine translation, sentiment analysis and information retrieval systems. The growth of digital text data has made NLP increasingly important for businesses and researchers alike.

Original Length: 79 words

Generated Summary:

Natural language processing (NLP) is a branch of artificial intelligence that helps computers understand, interpret and manipulate human language. NLP is used in applications such as chatbots, machine translation, sentiment analysis and information retrieval systems.

Summary Length: 30 words

The mini application successfully generated a concise extractive summary by scoring sentences based on normalized word frequencies and selecting the highest-scoring sentences, demonstrating a practical, end-to-end NLP pipeline.

Result

Thus the program to implement a develop a mini NLP application - an extractive text summarizer - that takes a paragraph as input and produces a concise summary by scoring and selecting has been executed successfully.